t07 · Palfrey — Tool ที่ใช้อ่านแบบ

ฉบับอธิบายง่าย · เน้นว่าแต่ละ tool ทำอะไร และตัวไหนเอาไอเดียมาจาก opentakeoff · เขียน 4 พ.ย. 69 · ตรวจจากไฟล์จริงใน repo

> **เล่าเป็นภาพ:** ระบบอ่านแบบทำงานเหมือนช่างถอดแบบหนึ่งคน คือ **เปิดแบบ → แยกว่าแต่ละหน้าคือหน้าอะไร → ตัดส่วนที่ต้องอ่านออกมา → อ่านค่า → เอาไม้บรรทัดมาเช็กระยะ → จดลงตาราง** ทุกขั้นมี tool ของตัวเอง บางขั้นใช้ AI อ่าน บางขั้นใช้โปรแกรมธรรมดา (CPU) ที่ไม่เดา และ t07 เปลี่ยนแค่ "ตัว AI กับคำสั่งที่มันได้รับ" ส่วน tool ที่เหลือไม่ต้องแตะ
> **สถานะ t07:** เตรียมแล้ว ยังไม่เทรน

สายอ่านแบบ 8 ขั้น



เอาอะไรมาจาก opentakeoff

opentakeoff คือโปรแกรมโอเพนซอร์สถอดปริมาณจากแบบ (Kentucky-ai บน GitHub) ไม่มีเรื่องเหล็กเสริม แต่มีหลักคิดเรื่อง "วัดให้แม่นและบอกที่มาของตัวเลข" ที่เรายืมมา **เฉพาะไอเดีย เขียนโค้ดเอง**

#	ไอเดียที่เอามา	อยู่ใน tool ไหนของเรา	สถานะ
1	**วัดจากเส้นจริงที่ซ่อนอยู่ใน PDF** ไม่ใช่วัดจากพิกเซลของรูป	`pdf-vector-sidecar.js` (เว็บดึงเส้น) + `vector_ruler.py` (worker ใช้เส้นนั้นเป็นไม้บรรทัด)	ทำแล้ว
2	**"มาตราส่วนคือด่านตรวจ ไม่ใช่ค่าเริ่มต้น"** ไม่เชื่อสเกลที่พิมพ์ในแบบ ถ้าไม่แน่ใจก็ไม่วัด	`pass3_measure.py` + `vector_ruler.py` รายงานอย่างเดียว ไม่แก้ค่า · ทิศแกนกำกวมปฏิเสธ · ต้องตรงสเกลมาตรฐาน	ทำแล้ว
3	**บอกให้ชัดว่ายอดรวมพึ่งค่าที่ยังไม่ได้ตรวจอยู่เท่าไหร่**	แถบเตือนหน้า BOQ (`renderReviewSummary` ใน `boq-summary.js`) โชว์ "มูลค่ารวมที่ติดธง + กี่ % ของยอดรวม" แทนนับแค่จำนวนแถว	ทำแล้ว 26 ก.ย.
4	จดที่มาของการวัดรายแผ่น (สเกล + วิธีวัด + ใครแก้ + ค่าเดิม)	อยู่ในแผนวิจัย 22 ต.ค. · ค้นโค้ดแล้ว **ยังไม่มี**	ยังไม่ทำ
5	คลิกเส้นกริด 2 เส้นแล้วพิมพ์ระยะ เพื่อตั้งสเกลเองในหน้าตรวจ	อยู่ในแผนวิจัยเช่นกัน · **ยังไม่มี** ในโค้ด	ยังไม่ทำ
–	โหมด raster ของเขา (ใช้กับภาพสแกน)	ไม่เอา: ปิดอยู่ใน build ปกติ และใช้เติมพื้นที่ ไม่ได้หาสเกล	ไม่เอา

เรื่องลิขสิทธิ์: ตอนเริ่มดู (26 ก.ย.) บันทึกเขียนว่า Apache-2.0 แต่เช็กซ้ำ 22 ต.ค. พบว่าตอนนี้เป็น AGPL-3.0-only (โค้ดถึง commit 84c7f48e ยังเป็น Apache) ทางที่เราเลือกคือเอาแต่ไอเดีย ไม่ก๊อปโค้ด

แต่ละ tool ทำอะไร (ขั้น 1–4 และตัวรับงาน)

เว็บเตรียมไฟล์ `drawing-purson.js` `pdf-vector-sidecar.js` เว็บ ไอเดียจาก opentakeoff

เปลี่ยน PDF เป็นรูปให้โมเดลดู และถ่ายสำเนา "เส้นจริง" ที่ซ่อนอยู่ใน PDF ไปด้วย

รูป: เรนเดอร์ที่ 200 dpi ซึ่งเป็นขนาดเดียวกับรูปที่ใช้สอนโมเดล โมเดลจะได้ไม่งงว่ารูปใหญ่หรือเล็กผิดจากที่เคยเห็น

เส้นจริง: PDF ที่ export จากโปรแกรม CAD ไม่ได้เก็บแค่รูป ข้างในยังจำพิกัดของทุกเส้นไว้เป๊ะๆ รวมถึงเส้นกริด (เส้นประยาวที่ลากผ่านกลางแปลน) ไฟล์นี้ดึงเส้นพวกนั้นเก็บเป็น `vectors_v1.json` แนบไปกับงาน เพื่อเอาไปเป็นไม้บรรทัดทีหลัง

ถ้า PDF เป็นภาพสแกน (ไม่มีเส้นจริง) จะข้ามหน้านั้นเฉยๆ ระบบไม่พัง · ที่ลองไว้: 22 จาก 53 เล่มเป็นเวกเตอร์ อีก 28 เล่มเป็นภาพสแกน

ตัวรับงาน `worker.py` รันในเครื่อง t07: ต้องแก้ path prompt

เหมือนผู้จัดการคิวงาน: หยิบงานจากคิว สั่งแต่ละขั้นตามลำดับ แล้วส่งผลกลับ

เว็บโยนงานเข้าคิวที่ Supabase → worker มาหยิบ → ส่งรูปให้ GPU อ่านทีละหน้า และเรียก tool CPU ในจังหวะที่ต้องใช้ → เก็บผลเป็นชุดไฟล์ raw-JSON ส่งคืนเว็บ

ข้อที่ต้องรู้: worker ประกอบ prompt ให้ **เหมือนตอนเทรนทุกไบต์** เพราะโมเดลเรียนมากับ prompt หน้าตาแบบนั้น ถ้าเปลี่ยนคำสั่งแล้วไม่ตรงกับตอนสอน ผลจะเพี้ยน

pass0 · แยกชนิดหน้า โมเดล Destrier AI · GPU t07 เปลี่ยนผล

ดูทีละหน้าแล้วบอกว่า "หน้านี้คือหน้าอะไร"

เช่น ผังฐานราก ผังคาน รูปตัด ตารางสเปก จะได้ส่งแต่ละหน้าไปถามคำถามที่ถูกเรื่อง ไม่เอาผังคานไปถามเรื่องฐานราก

pass1 · ตัดครอป `organize.py` CPU

ตัดแบบเป็นชิ้นให้พอดีมือโมเดล

แบบหนึ่งแผ่นมักมีหลายวิวอยู่รวมกัน ถ้าส่งทั้งแผ่น ตัวหนังสือจะเล็กจนอ่านยาก ขั้นนี้ตัดเป็นครอปทีละวิว ให้ขนาดตัวอักษรใกล้เคียงรูปที่ใช้สอน

ถ้าขั้นนี้พัง ระบบส่งเต็มหน้าแทนแล้วอ่านต่อ ไม่หยุดงาน

pass1.5 · ชี้เป้า `cv_scan.py --manifest` CPU · OpenCV

ให้โปรแกรมหาภาพแบบเทมเพลตช่วยชี้ก่อนว่า "ตรงนี้น่าจะมีฐานราก/เสา/คาน"

เอาภาพสัญลักษณ์ตัวอย่างไล่เทียบกับแบบ เจอตรงไหนก็แนบเป็น "คำใบ้" ต่อท้าย prompt ของ plan_footing / plan_beam / plan_slab ให้โมเดลอ่านโดยมีเป้าหมายอยู่ในมือ

แต่ละ tool ทำอะไร (ขั้น 5–8)

pass2 · อ่านค่า โมเดล Destrier `AI · GPU` `t07 เปลี่ยนผล (ตัวหลัก)`

ตัวอ่านจริง: ถามทีละหน้า ทีละหัวข้อ แล้วได้คำตอบเป็น JSON

หัวข้อคือฐานราก เสา คาน พื้น และ **gridline** (เส้นกริดแปลน: ตำแหน่ง A, B, C / 1, 2, 3 และระยะห่างระหว่างเส้น) ขั้นนี้อ่าน gridline ก่อนเสมอ แล้วเอาผลไปใส่ใน prompt ของทุกหน้าที่เหลือ โมเดลจะรู้ว่าเสาและคานแต่ละตัวอยู่ที่กริดไหน

ปัญหาที่ t07 แก้อยู่ตรงนี้: gridline ของบ้านใหม่วนพ่นชื่อบ้านเก่าจนชนเพดาน 9,000 token

pass2.5 · เก็บตก `cv_scan.py --pass25` `CPU · OpenCV`

เก็บสัญลักษณ์ที่แม่แบบกลางจับไม่ได้

บางเล่มวาดสัญลักษณ์หน้าตาไม่เหมือนชุดอื่น ขั้นนี้ไล่เก็บส่วนที่ pass1.5 พลาด แล้วเก็บเป็นไฟล์แยก ไม่ปนกับผลอ่านหลัก

pass3 · ไม้บรรทัด `pass3_measure.py` `vector_ruler.py` `CPU` ไอเดียจาก opentakeoff

เอาไม้บรรทัดมาเช็กว่าความยาวที่โมเดลอ่านได้ ตรงกับแบบจริงไหม

pass3_measure.py: ใช้ตำแหน่งหมุดที่โมเดลอ่านได้ในรูป เทียบกับระยะกริด คำนวณว่า "1 เมตรเท่ากับกี่พิกเซล" แล้วเอาไปวัดคาน

vector_ruler.py: ทำแบบเดียวกัน แต่ใช้เส้นกริดจริงใน PDF เป็นไม้บรรทัด ไม่ต้องพึ่งโมเดล วัดผังคานล้วนได้ (ส่วน `pass3_measure.py` วัดไม่ได้เลยในงานจริง 0 จาก 9 หน้า) และตรวจ grid master ด้วยว่าระยะเพี้ยนเป็นเท่าไหม เช่น อ่าน 3.00+3.00 เป็น 0/6/12 ซึ่งยาวเป็น 2 เท่า

กติกา: ไม่เชื่อสเกลที่พิมพ์ในแบบ (บ้าน 01 พิมพ์ 1:100 แต่จริงคือ 1:141.4 เพราะย่อ A3 ลง A4) · ทิศแกนกำกวมให้ปฏิเสธ ไม่เดา · **รายงานอย่างเดียว ไม่แก้ค่าที่อ่านมา**

ทดลองกับเฉลย 98 หน้าเวกเตอร์: วัดได้ 68 หน้า (ผังคาน 38 จาก 47) · **ยังไม่เคยรันกับงานจริงผ่านการ์ด**

เว็บแปลงผล `raw-extraction-adapter.js` `raw-extraction-import.js` `เว็บ` `t07: แก้ 1 บรรทัด`

ล่ามแปลคำตอบของโมเดลให้เป็นชิ้นส่วนที่คำนวณราคาได้

แปลง JSON เป็นเสา คาน ฐานราก พร้อมสเปกเหล็ก แล้วส่งเข้า `runPipeline()` ได้ BOQ และตารางดัดเหล็ก (BBS) ความยาวคานอิงระยะระหว่างกริดเป็นหลัก ถ้าหาไม่ได้จะติดธงเตือนชัดเจน **ไม่เดาเงียบๆ** และของที่อ่านไม่ชัดก็ติดธงให้วิศวกรตรวจ

บรรทัด 516 ตอนนี้หา grid master ด้วย `source_pages` ซึ่ง t07 ไม่ส่งแล้ว ต้องแก้ให้ดูแค่ `pattern === 'gridline'`

Tool ที่ต้องใช้ทำ t07

งาน	Tool ทำอะไร	สถานะ	ผ่านเมื่อ
1) เตรียมข้อมูลสอน ฟรี	`sanitize_labels.py` ลบชื่อบ้านเก่าและประวัติการแก้ออกจากเฉลยที่ใช้สอน ไม่ให้โมเดลท่องจำ `build_t07.py` สร้างชุดข้อมูลสอน 4 ชุด (ก็อปจาก t05) `prompts/` คำสั่งที่ตัดช่อง `source_image` ซึ่งโมเดลมองไม่เห็นออก `onstart.sh` สคริปต์เปิดการ์ดเช่า ตรึงเวอร์ชันโปรแกรมกันเปลี่ยนเงียบๆ	มีแล้ว ตัวแรก (selftest ผ่าน) ยังไม่ทำ อีก 3 ตัว	ชื่อบ้านเทรน / บันทึกสคริปต์ / `source_*` = **0** · element ไม่ลด · ตัวอย่างยาวสุดไม่เกิน MAX_LENGTH
2) ทดลองบนการ์ดถูก การ์ดถูก	เทรน 10 step → เซฟ → โหลดกลับ → ยิง 1 ภาพ `check_expert_lora` ตรวจว่าชั้น MoE โหลดถูกท่าครบ 80 ตัว	มีแล้ว	ขึ้น **"ทั้ง 80 ตัว grouped_by_expert"**
3) เทรน + รวม เสียเงิน	เทรน 4 ชุดแบบ t05 `merge_adapters_soup.py` รวมผลเทรนทั้ง 4 ชุดเป็นตัวเดียว `test_soup_layout.py` ตรวจว่าไม่ปนผิดท่า	มีไฟล์ ยังไม่ได้รันซ้ำ	`test_soup_layout.py` ผ่าน · **ถามมะขามก่อนเช่าการ์ด**
4) merge + เสิร์ฟ	`merge_lora_to_base.py` ฝังผลเทรนเข้าโมเดลฐาน · `verify_merge.py` ตรวจ 3 ชั้น (A/B/C) `presentation.py` preset `destrier-vllm` + `serve_purson.py` เปิดเซิร์ฟเวอร์โมเดลบนการ์ดเช่า (vLLM เร็วกว่าตัวเดิมหลายสิบเท่า วัด 3 ต.ค.) · ด่าน D เทียบความมั่นใจคำตอบ (logprobs) กับโมเดลที่ merge	มีไฟล์	A/B/C ผ่าน · ด่าน D ตรงกัน
5) วัดผล	`bench_house.py` ให้โมเดลอ่านทั้งบ้านจริง แล้วจดเวลา / token / ตัวอักษรจีนของทุกคำขอ ใช้บ้าน 83b8e52c + บ้านจากเน็ต 3 หลัง · `micro --repeat 4` ยิงหน้าเดิมซ้ำดูความแกว่ง	มีไฟล์	ผ่านตารางข้างล่างครบทุกแถว

เกณฑ์ผ่านสุดท้าย

ฐานเทียบ = Destrier ตัวซ่อมบน vLLM (proof/04)

วัดอะไร	ฐานตอนนี้	t07 ต้องได้
recall / precision มาร์ค (บ้าน 83b8e52c)	`0.79–0.83 / 0.83–0.85`	**`≥ 0.79 / ≥ 0.83`**
บ้านเน็ต: gridline ชนเพดาน 9,000 token	ชน 3 ครั้งใน 6 รอบ	**0 ครั้ง (3 หลัง × 3 รอบ)**
ชื่อบ้านเทรนโผล่ในผลอ่าน	มี	**0**
ตัวอักษรจีน (`cjk`)	`0`	**`0`**
plan_beam หน้า 20 ยิงซ้ำ 4 รอบ (เฉลย 59)	22–56	**ช่วงแคบลง · ต่ำสุด ≥ 45**

จุดที่ต้องจัดการตอน deploy

ข้อที่ workflow ยังไม่ได้เขียน: worker ผูก prompt กับ path ของ tool CPU ไว้ด้วยกัน

อ่านจากโค้ด ยังไม่ได้ลองรัน

- `worker.py` อ่าน prompt จาก `PURSON_PROMPTS_DIR` (ตอนนี้ `worker_config.json` ชี้ `t04_Purson`) และหา `organize.py` กับ `cv_scan.py` จาก `PROMPTS_DIR.parent` (บรรทัด 140–141)
- **ถ้าไม่เปลี่ยน:** โมเดลที่เทรนด้วย prompt ใหม่จะได้ prompt เก่า (ยังมี `source_image`) ไม่ตรงกับตอนเทรน
- **ถ้าชี้ไป** `t07_Palfrey/prompts/` ตรงๆ: path ของ `organize.py` / `cv_scan.py` จะพัง เพราะ parent เปลี่ยน
- **ทางที่น่าจะง่ายสุด:** แยก path ของสอง tool นั้นออกจาก `PROMPTS_DIR` แล้วค่อยชี้ prompt ไปที่ t07

ที่เหลือหลังผ่านครบ

1. แก้ `raw-extraction-import.js` บรรทัด 516 ให้เหลือแค่ `pattern === 'gridline'`
2. push โมเดล merged ขึ้น HF (ต้องมี HF write token — ณ 3 ต.ค. ยังไม่มี) แล้วชี้ preset `destrier-vllm` ไป repo ใหม่
3. temperature ใน `worker.py` ตอนนี้ 0.7 ต้องวัดใหม่หลังเทรน และวัดทั้งบ้านเสมอ

กติกาเหล็กของรอบนี้

- ด่านไหนไม่ผ่านให้หยุด จดเลขที่วัดได้ ห้ามแก้ข้ามด่าน · ห้ามลด MAX_LENGTH · วัด sampling ด้วย **ทั้งบ้าน** ห้ามตัดสินจาก micro
- ห้ามแก้ไฟล์ใน `json_แก้ไขแล้ว/` · เช่าการ์ดเทรนต้องถามมะขามก่อนเสมอ · เทรนจบแล้วคืนการ์ดและยืนยัน 0 instance

---

สรุปจาก `t07_workflow.md`, `worker.py`, `vector_ruler.py`, `pdf-vector-sidecar.js`, `boq-summary.js`, ไดอารี่ 26 ก.ย. และเอกสารวิจัย tools-2026-10 (REPORT_wave1 / LICENCE_RECHECK) · ตัวเลขฐานมาจาก proof/04 และ proof/05 ของ t05_Destrier ไม่ได้วัดใหม่ในเอกสารนี้